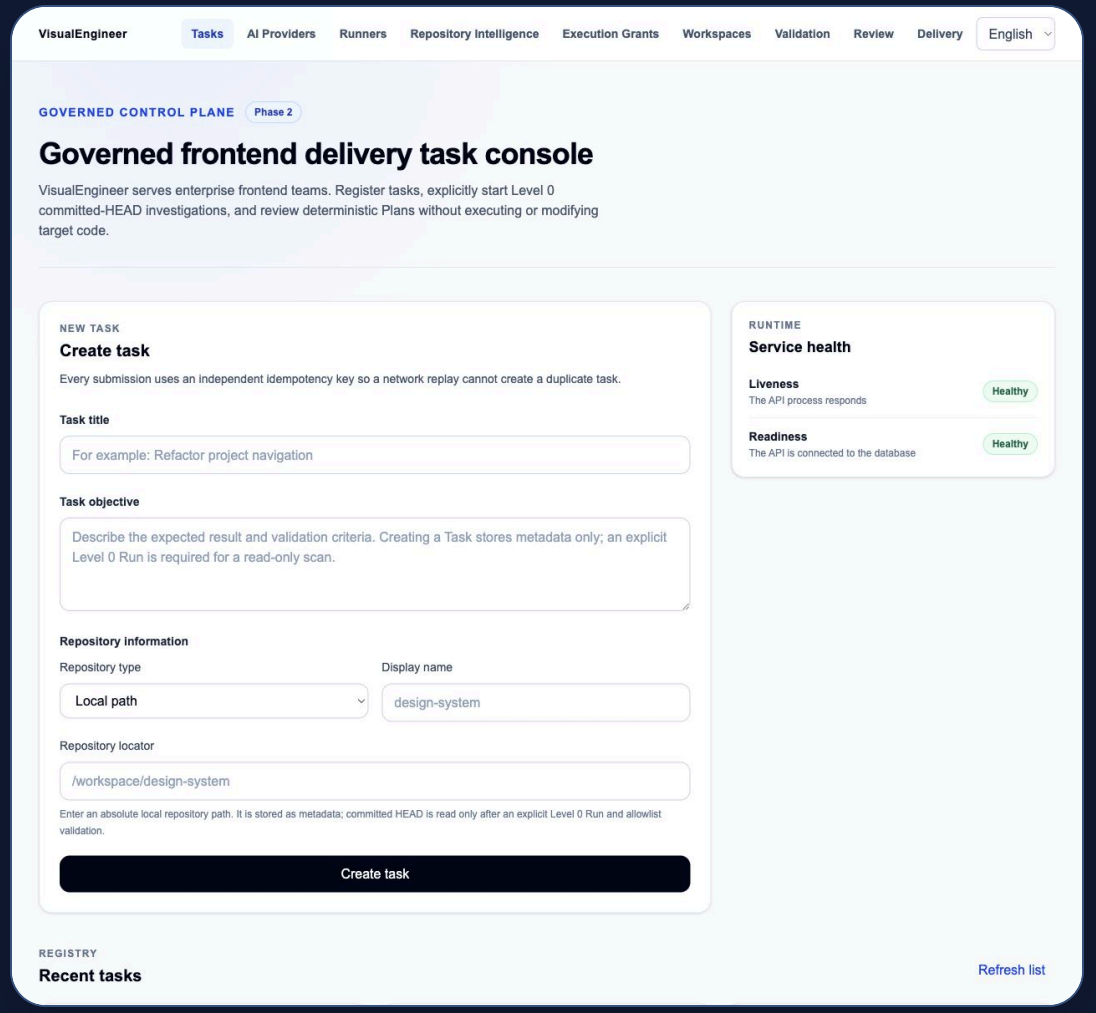
面试作品集

VISUALENGINEER

受治理的 AI 设计工程与交付。

一个本地单用户系统，将 AI 规划、授权、执行与证据保持为彼此独立、可审查的决策。

工程案例研究
github.com/dankk0220abc-prog/visual-engineer



问题

AI 可以提出实施建议，但不应自动获得修改代码仓库的权限。

有价值的计划**不是**修改工作区、运行命令、批准评审或交付结果的权限。

01 / 分离

规划与权限分离

Provider 输出是类型化数据，进入受控生命周期，而不是直接成为工具调用。

02 / 限定

执行受 Grant 约束

只有获得独立范围授权，Runner 才可在 fixture 工作区中执行合格任务。

03 / 保留

如实保留不确定性

模糊的外部结果保持为 `outcome_unknown`，不会自动重试。

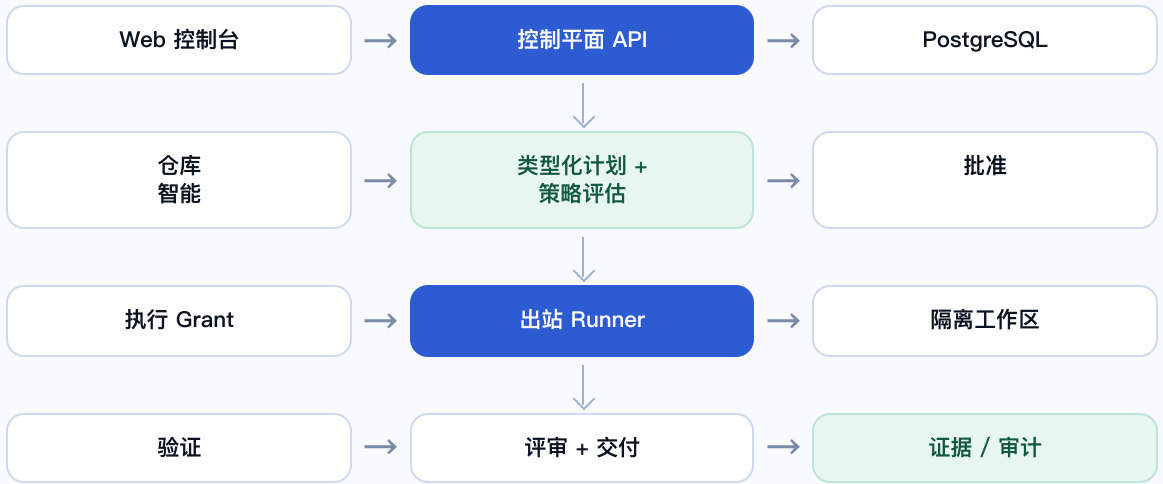
04 / 投影

安全地呈现证据

可审查事实不包含密钥、原始流量、完整上游 ID 或账单材料。

系统架构

围绕受限 AI 提案构建的控制平面。



关键边界

Provider Layer 可通过控制平面返回类型化规划产物，但不拥有任何直接的 Runner 或工作区权限。

职责概览

W **Web 控制台**

浏览器优先的呈现、工作流和证据界面。

A **控制平面**

生命周期、策略、权限转换和安全投影。

R **本地 Runner**

封闭类型化任务与 Grant 的出站消费者。

E **证据**

适合评审的审计事实，而非秘密归档。

受治理的工作流

计划批准不等于执行 Grant。

Provider 提案

类型化 Plan 经受治理路径生成，并由系统策略进行评估。

Plan 已生成

!=

权限边界

成功的提案不会决定批准、签发 Grant、授予 Runner 权限、创建评审或授权交付。

需要人工决策　需要独立范围

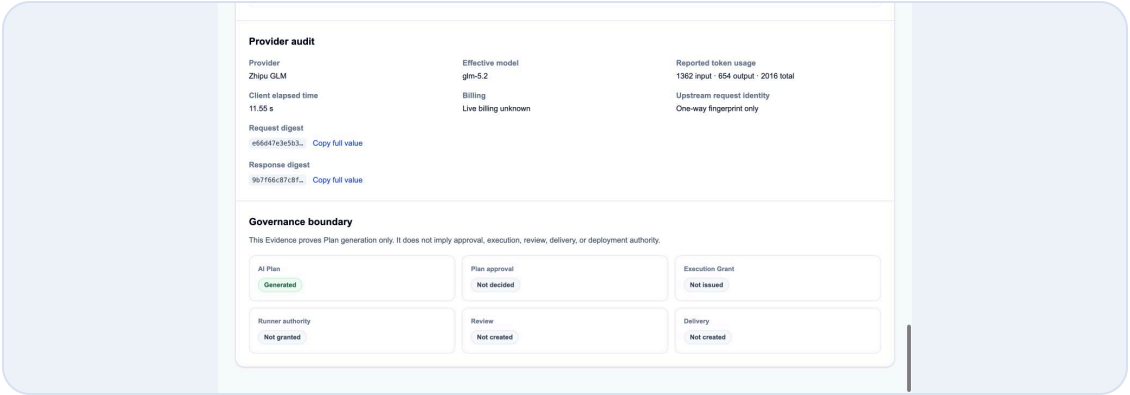
Runner 工作

只有获得独立 Grant 的合格类型化任务，才可到达 Runner 持有的 fixture 工作区。

需要 Grant

Provider 无法执行的操作

签发 Grant、运行 shell 命令、修改工作区、批准评审、授权交付、合并或部署。



PROVIDER 层与 BYOK

支持、连接和 Live 验证是彼此独立的事实。

Deterministic Fixture 是默认选项

离线、无网络、不可计费。它是产品演示中最安全、可复现的模式。

无需凭据　Fixture 已验证

唯一 LIVE 已验证 PROVIDER

Zhipu / GLM-5.2

其他显示的适配器

已支持不等于已完成 Live 验证

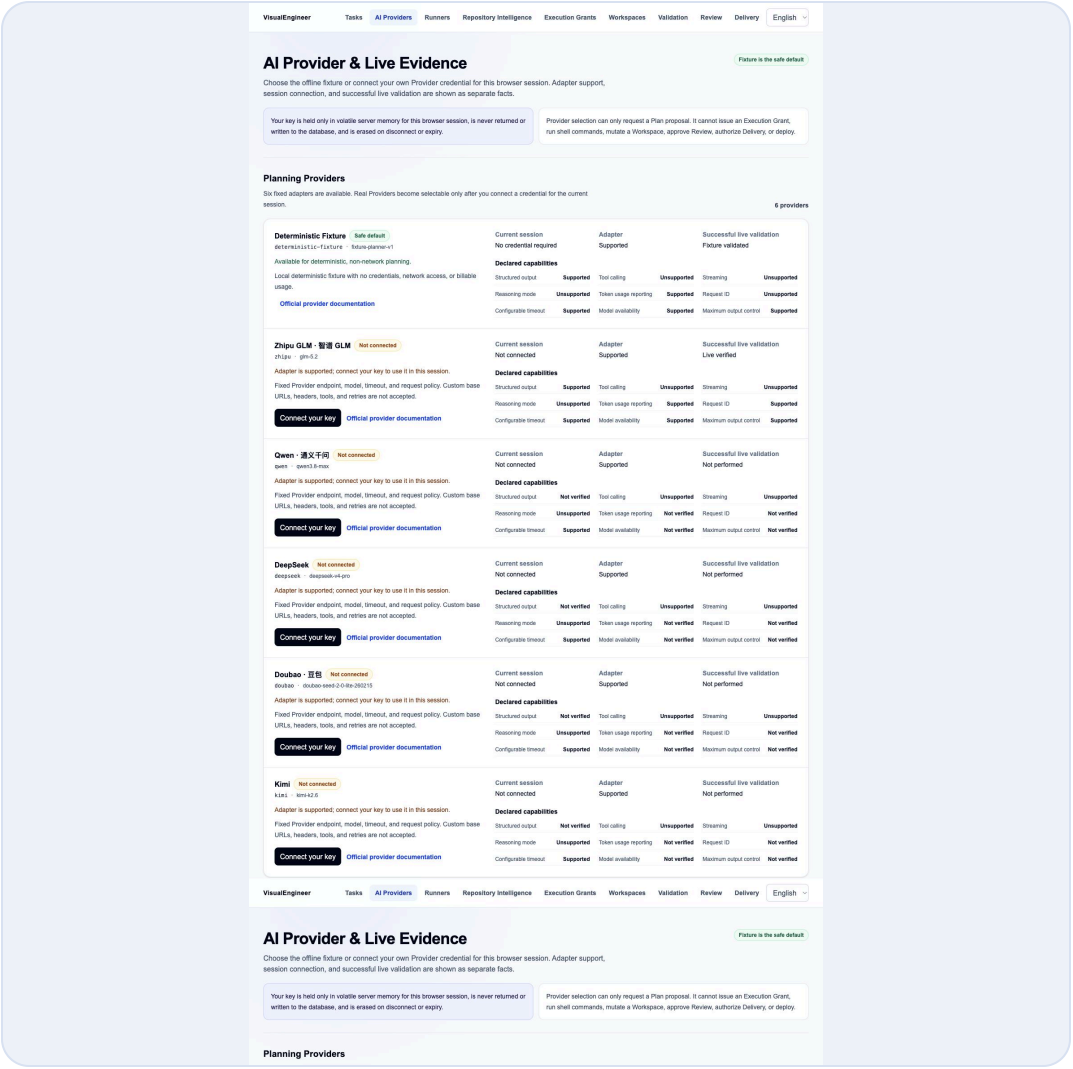
BYOK 处理方式

当前会话的易失性服务器内存

配额边界

普通用户不会回退使用维护者凭据

> 凭据不会被返回或持久化，并会在断开、到期或重启时清除。这不被表述为商业级多租户密钥库。



已封存的 LIVE 验证

一次受治理的 Zhipu / GLM-5.2 调用成功完成。

关联标识

3a9c354f7a58

安全证据关联标识

报告用量

2,016 tokens

1,362 input + 654 output

测得延迟

11.55 s

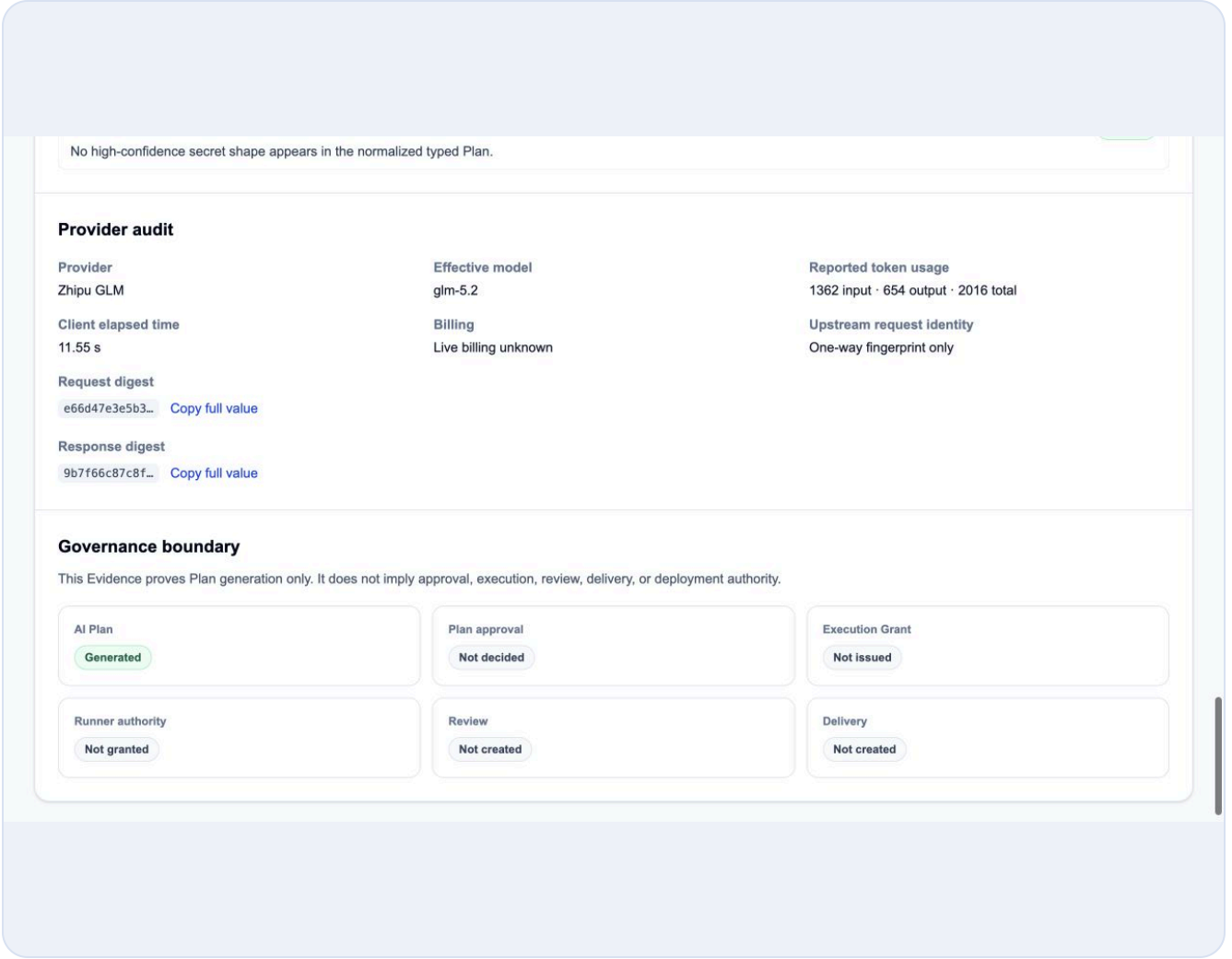
Provider 11,543 ms；客户端总计 11,553 ms

账单

live_billing_unknown

不主张金额或账单确定性

生命周期：succeeded　类型化 Plan：3 项操作　8 项策略检查通过



失败工程故事

派发后的未知状态是事实，不应被粉饰为简单的失败标签。

第 01–02 次

传输不确定性

派发后，系统无法如实判定 Provider 的实际结果。

outcome_unknown

第 03 次

响应路径诊断

如实分类解码问题，同时保留早期不可变历史。

不自动重试

窄范围修复

修复兼容性，而非扩大权限

增加安全传输可观测性、身份内容编码协商和解码响应归一化。

受限变更

第 04 次

独立的成功事件

第四次受治理调用成功，但仍未产生下游执行权限。

succeeded

为何不自动重试？模糊请求可能已到达可计费或有副作用的外部系统。后续尝试必须作为新的、受控事件。

RUNNER 与工作区隔离

模型绝不获得不受限制的 shell 权限。



在边界上保留的控制

- ✓ 封闭类型化操作，而非任意命令
- ✓ 精确修订、Runner 持有的 fixture 工作区
- ✓ 独立 Runner 身份和出站协议
- ✓ 任务结算周围的 Lease / fencing
- ✓ 固定验证 Profile 和受限证据
- ✓ 不存在 Provider 直达 Runner 的权限路径

源仓库、远程 Git、合并、发布和部署权限均被明确排除在本演示路径之外。

修复原子性边界，而不是用重试掩盖半写入。

观察到的 Red 基线

嵌套的 Runner 创建在子 Validation Run 绑定完成前提交。第二个事务可以看到局部状态，并与子行竞争。



10 / 10 次确定性的真实死锁

受控修复

将 Runner 创建保持在外层 Validation 事务中。竞争事务会在进入部分绑定子记录窗口前等待。



10 / 10 次受控通过 – 零未绑定子记录 – 不盲目重试。

完成声明由证据账本支撑，而不是一句绿色状态。

声明与证据纪律

只有来源已知且可安全投影的声明才会被接受。未知仍保持未知。

源身份

Git SHA 与 tree 确立已封存的仓库状态。

验证

CI、测试、浏览器证据和数据库证明回答不同问题。

Provider 生命周期

prepared、dispatching 与终态分类保留外部不确定性。

无秘密证据

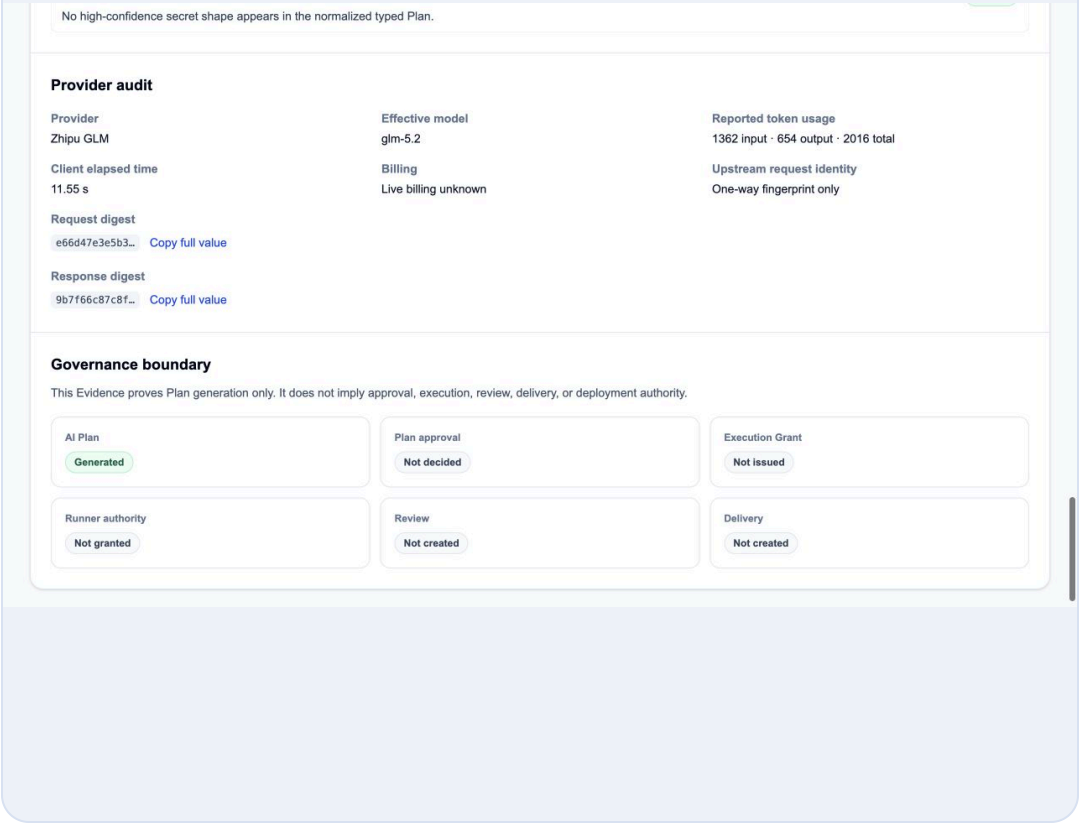
公开安全材料排除密钥、私有载荷、账户和账单记录。

失败历史

早期模糊尝试是持久事实，不会被追溯性改写为成功。

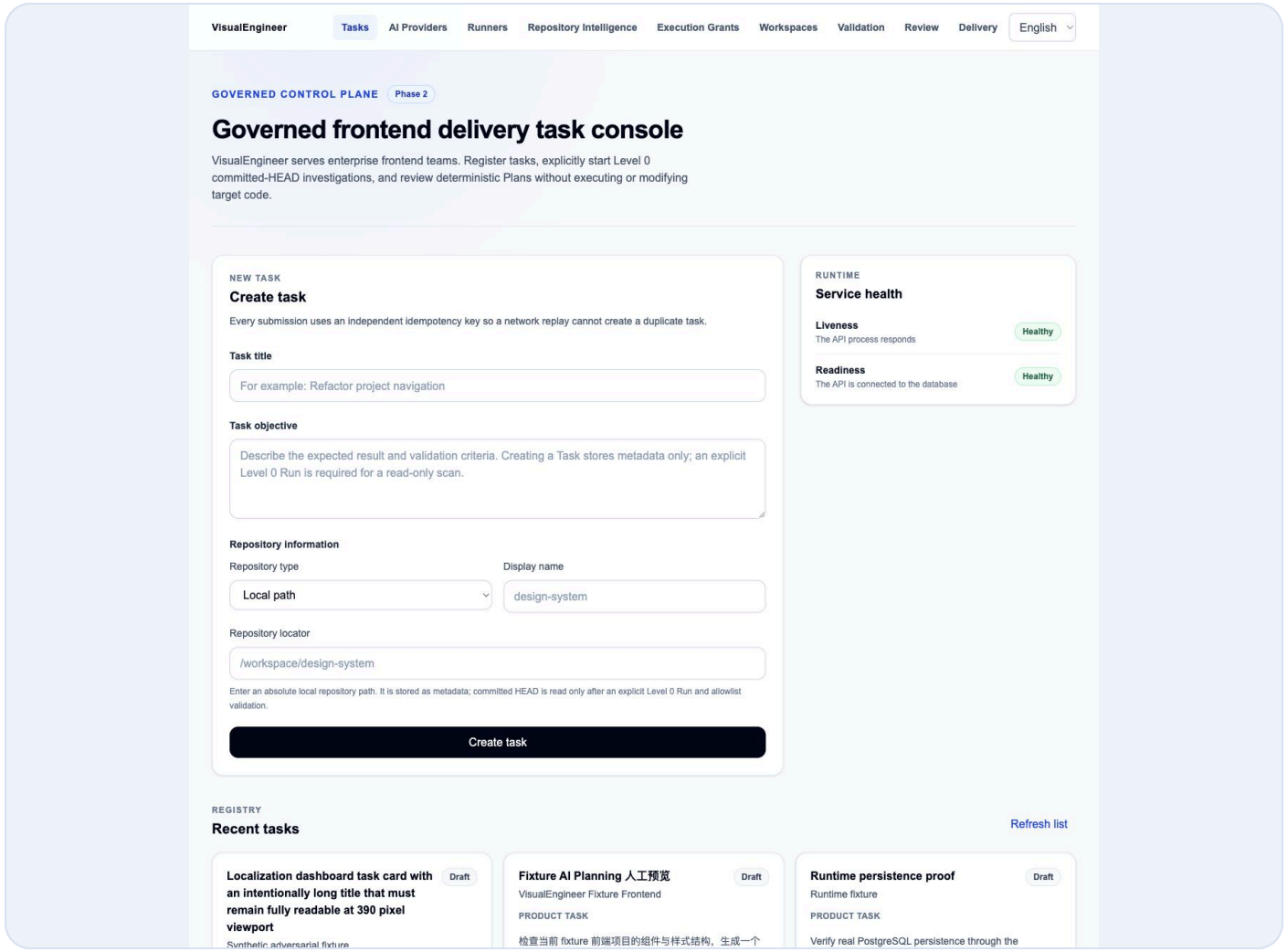
范围限制

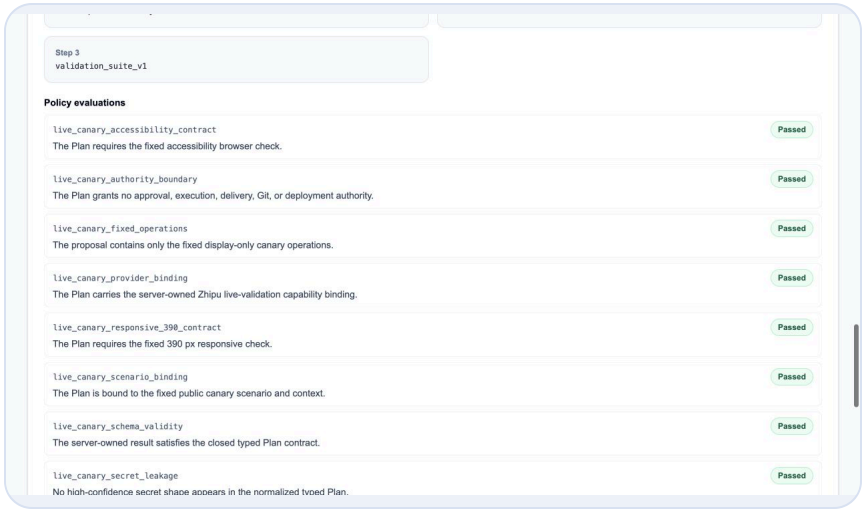
本地演示证明不被重新表述为生产部署或客户使用。

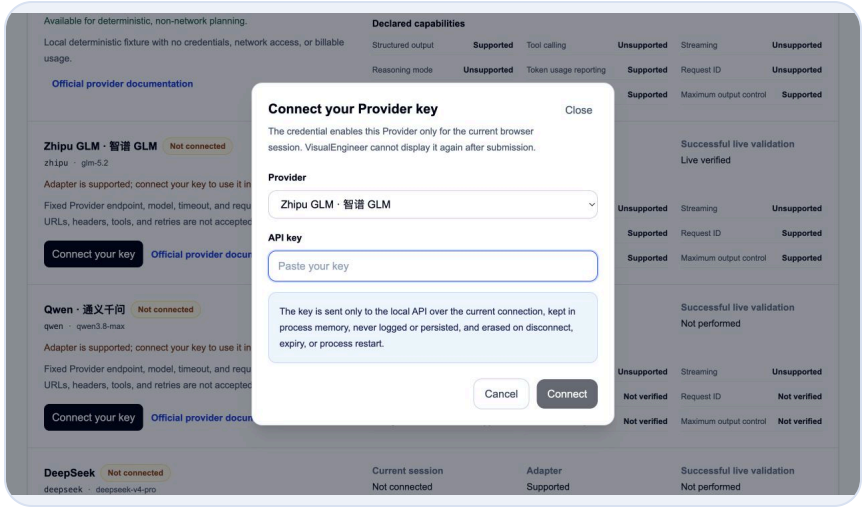


产品体验

让治理边界可见的浏览器优先控制台。







zh-CN / en-US | 响应式体验 | 类型化 Plan Evidence | Governance Evidence | Provider 状态 | Fixture / Live 区分 | BYOK 入口

工程技术栈

从浏览器 UI 到持久化与证明的类型化边界。

WEB 控制台

Next.js App Router
React
TypeScript strict
Tailwind CSS
next-intl

控制平面

Python
FastAPI
Pydantic
SQLAlchemy
Alembic

状态与执行

PostgreSQL
独立本地 Runner
封闭类型化任务协议
隔离 fixture 工作区

质量与证据

Vitest
Playwright
axe
Ruff 与 mypy
Git 与 CI 证据

技术栈的重要性在于，每一条边界都由具体契约表示：客户端的 TypeScript、API 边缘的 OpenAPI、类型化 Python 服务，以及保障持久状态的 PostgreSQL 约束。

系统边界的责任始终由人承担。

我负责的内容

产品	问题定义、工作流、界面优先级与验收标准。
架构	权限边界、生命周期规则、类型化契约与风险决策。
验证	证据策略、测试预期、失败解读与最终验收。
说明	能够讨论架构、状态机、故障、权衡与范围限制。

ChatGPT + Codex 如何加速工作

在明确约束与审查下，AI 协助加速仓库调研、实现、测试、文档及 Git / CI 工作。

本作品集不作的声明

它不声称每一行代码都由人工编写，也不会将产品选择、安全边界、验证或最终验收的责任转移给模型。

> 最有价值的面试讨论不是“AI 写了它”，而是系统为何让权限、不确定性和证据清晰可见。

范围与下一步

清晰的边界，比夸大的声明更适合作为演示。

github.com/dankk0220abc-prog/visual-engineer

- 主要开发已完成；最终封存证据包含一次成功的 Zhipu / GLM-5.2 Live 验证。
- 其他适配器不得被表述为已完成 Live 验证。
- Fixture 仍是最安全、可复现的演示模式。
- 仅为本地单用户证明：不主张生产部署、客户、收入、用户规模或性能提升。
- BYOK 是会话范围的易失性处理，而非商业级多租户密钥库。

面试讨论重点：Provider 生命周期、PostgreSQL 不变量、Runner 边界或证据策略。